CineTrack: Mobile Movie Catalog and Geospatial Cinema Mapping Technical Report

André Moreira (1240567) and Afonso Sobreiro (1241013)

ISEP, Instituto Superior de Engenharia do Porto,
Rua do Dr António Bernardino de Almeida 431, 4249-015 Porto, Portugal
{1240567,1241013}@isep.ipp.pt
https://www.isep.ipp.pt

Abstract. This technical report documents the design, architecture, and implementation of **CineTrack** (*ProjectDroid*), a native Android mobile application developed for the Software Development and Mobile Systems (*DSSMV*) course unit within the Informatics and Telecommunications Engineering degree at ISEP. CineTrack unifies movie catalog discovery, personal collection management, and location-based cinema mapping into a responsive mobile experience. The report details the system's pure Java 21 domain model, technical glossary, class diagram, UI navigation flow, use case specifications, and design sequence diagrams. Furthermore, it describes the implementation of critical mobile engineering concerns: non-blocking asynchronous REST communication with The Movie Database (TMDB) and Foursquare Places APIs via Retrofit 2, realtime Cloud Firestore synchronization, hardware sensor integration (accelerometer-based *Shake to Suggest* and native GPS positioning), OpenStreetMap (OSMDroid) interactive mapping with search radius overlays, and background battery preservation. Finally, automated JUnit 5 test suites and verification results on physical hardware are presented, demonstrating compliance with mobile engineering best practices and architectural standards.

Keywords: Android, Java 21, MVC Architecture, Cloud Firestore, Retrofit, Foursquare Places API, OSMDroid, Accelerometer Sensor, TMDB, DSSMV.

1 Context and Project Scope

The proliferation of digital streaming platforms has created an increasingly fragmented landscape for cinema enthusiasts seeking where specific titles are available for viewing or looking for physical screening rooms nearby. The **CineTrack** mobile application addresses this challenge by providing an integrated, responsive solution that merges remote cinematic catalogs, cloud-synced personal lists, hardware sensors, and geospatial mapping.

Developed within the academic framework of the Software Development and Mobile Systems (*DSSMV*) course unit in the Informatics and Telecommunications Engineering degree at the Instituto Superior de Engenharia do Porto (ISEP), CineTrack is designed to embody core mobile software engineering practices. Developing robust software for mobile devices imposes strict architectural constraints due to limited physical resources, variable network connectivity, and the imperative requirement of maintaining graphical interface (*UI Thread*) responsiveness. To prevent *Application Not Responding* (ANR) dialogs and excessive battery depletion, CineTrack strictly follows four foundational mobile principles:

1. **Model-View-Controller (MVC) Separation:** Pure Java 21 domain entities isolated from the Android SDK, enabling fast, isolated unit testing without requiring emulator mocks.
2. **Non-Blocking Asynchronous I/O:** All network requests (Retrofit 2) and persistence updates (Cloud Firestore and SharedPreferences) are executed asynchronously off the main thread.
3. **Sensor Lifecycle Discipline:** Hardware listeners for the accelerometer and GPS are registered exclusively in `onResume()` and immediately detached in `onPause()`.
4. **Density-Independent UI Design:** Interface layouts in XML rely strictly on density-independent pixels (`dp`) and scale-independent pixels (`sp`).

2 Analysis: Requirements, Domain Modeling, and Glossary

2.1 Project Scope and Implementation Matrix

The CineTrack project was developed collaboratively using a structured Git feature branch workflow, segregating tasks into two parallel development branches (`andre-1240567` and `afonso-1241013`) before integration into the `main` branch via peer-reviewed Pull Requests. Table 1 summarizes the implementation status and automated test coverage across all 11 User Stories.

Table 1: CineTrack Implementation and Testing Matrix

ID	User Story Description	Author	Status	Tests
US1	[Auth] User Registration and Sign-In	André (1240567)	100% Complete	Manual / Live
US2	[CRUD] Create and Manage Movie Lists	André (1240567)	100% Complete	3 Unit Tests
US3	[CRUD] Add Movies and Submit Ratings	André (1240567)	100% Complete	10 Unit Tests
US4	[API] TMDB Search and Streaming Providers	Afonso (1241013)	100% Complete	7 Unit Tests
US5	[Map] Nearby Cinemas (GPS + OSMDroid)	Afonso (1241013)	100% Complete	7 Unit Tests
US6	[Sensor] Shake to Suggest (Accelerometer)	Afonso (1241013)	100% Complete	Hardware / Live
US7	[Auth] Profile Management and Stats Cards	André (1240567)	100% Complete	8 Unit Tests
US8	[Persistence] Local Preferences (apply)	Afonso (1241013)	100% Complete	13 Unit Tests
US9	[CRUD] Local Collection Filter and Sort	André (1240567)	100% Complete	10 Unit Tests
US10	[API] Official YouTube Video Trailers	Afonso (1241013)	100% Complete	4 Unit Tests
US11	[Social] External Movie Card Sharing	André (1240567)	100% Complete	8 Unit Tests
Additional DTO / API Integration Tests		Team	100% Complete	4 Unit Tests
Total	**All 11 User Stories Fully Implemented and Merged**			**74 Passing**

2.2 Conceptual Domain Model

The conceptual Domain Model defines the business entities and structural relationships of CineTrack independent of technological frameworks. As illustrated in Figure 1, the domain revolves around the central concepts of authenticated users, personal movie lists, saved movies, watch states, ratings, and physical cinema points of interest.

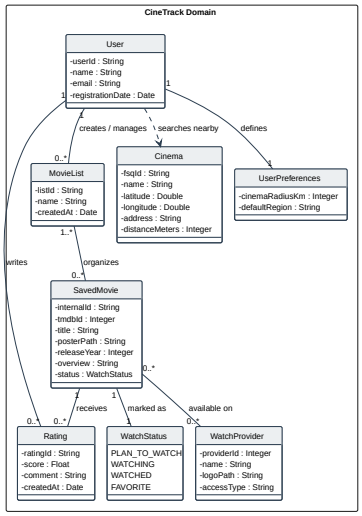


Fig. 1: CineTrack Conceptual Domain Model

The domain model enforces several essential business invariants:

- **User and MovieList Ownership:** A `User` uniquely owns zero or more `MovieList` entities. Each list contains an arbitrary number of `SavedMovie` instances.
- **WatchStatus Progression:** Every saved movie is categorized under a strict enumeration: `PLAN_TO_WATCH`, `WATCHING`, `WATCHED`, or `FAVORITE`.
- **Rating Interval:** A `Rating` score must strictly satisfy $1.0 \leq \text{score} \leq 5.0$. Values outside this range throw checked `InvalidRatingException` instances.
- **Geospatial Radius Bounds:** The user's search radius for cinemas must lie within $1 \leq \text{radius} \leq 100$ kilometers.

2.3 Technical and Domain Glossary

Table 2 establishes the standard technical and domain terminology used throughout the CineTrack documentation and implementation.

Table 2: CineTrack Technical and Domain Glossary

Term/Expression (EN)	**Termo (PT)**	**Definition/Description**
ANR	Falha por Inatividade	Application Not Responding dialog triggered by Android when the UI Thread remains blocked for more than 5 seconds.
Cinema	Sala de Cinema	Physical point of interest possessing latitude, longitude, formatted address, and distance, queried from Foursquare Places API.
Explicit Intent	Intent Explícito	Navigation message explicitly designating an internal Activity class within the application package.
Foursquare Places API	API Foursquare	REST service providing geospatial POI search by geographic coordinates and category (Cinema category `10024`).
Implicit Intent	Intent Implícito	Abstract action delegated to the Android operating system to resolve via external installed applications (e.g., Google Maps, YouTube).
MovieList	Lista de Filmes	Personalized user collection stored in Cloud Firestore (e.g., "Favorites", "Watchlist").
MVC	Padrão MVC	Architectural pattern separating pure Java Model entities, declarative XML Views, and Activity Controllers.
OSMDroid	OpenStreetMap SDK	Open-source Android library for rendering map tiles, circle overlays, and clickable marker pins without paid API keys.
Rating	Avaliação	Quantitative score in the interval $[1.0, 5.0]$ and optional qualitative review text.
Retrofit 2	Cliente Retrofit	Type-safe HTTP client abstracting asynchronous REST API invocations via `.enqueue()` and Gson serialization.
SavedMovie	Filme Guardado	Domain representation of a movie linked to a list, containing TMDB identifier, title, release year, poster URL, and watch status.
ShakeDetector	Detetor de Agitação	Custom `SensorEventListener` detecting deliberate multi-peak accelerations ($> 2.7g$) while ignoring incidental vibrations.
SharedPreferences	Preferências Locais	Lightweight key-value persistence. Writing operations strictly use asynchronous `.apply()` to avoid UI Thread locking.
TMDB	The Movie Database	External REST service supplying cinematic catalog discovery, streaming watch providers by country, and official trailers.
UI Thread	Thread Principal	Android's main execution thread responsible for screen rendering and touch events; must never execute blocking I/O.
WatchProvider	Fornecedor de Streaming	Licensed digital streaming service (e.g., Netflix, HBO Max, Disney+) available in the selected geographical region.

2.4 Global Use Case Architecture

Figure 2 presents the high-level Use Case Diagram synthesizing the four primary functional modules: *Module 1: Identity and Session*, *Module 2: Catalog and Multimedia*, *Module 3: Personal Lists and Ratings*, and *Module 4: Geolocation and Cinemas*, alongside external cloud integrations (Firebase, TMDB, Foursquare Places API) and device hardware sensors.

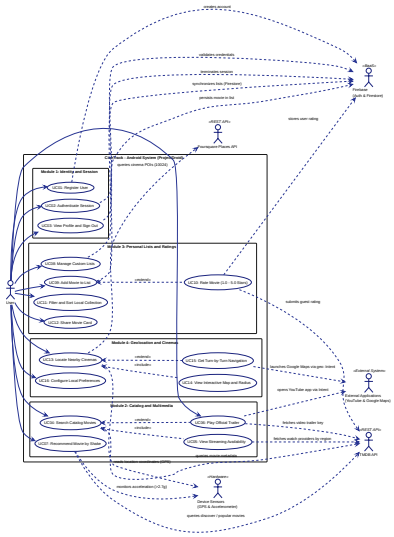


Fig. 2: CineTrack Global Use Case Architecture

3 Design: Architecture, Navigation Flow, and Realization

3.1 Model - Class Diagram

Figure 3 depicts the core class diagram of the pure Java 21 domain layer located in `pt.isep.dssmv.projectdroid.model`. The domain layer maintains strict independence from all Android SDK packages.

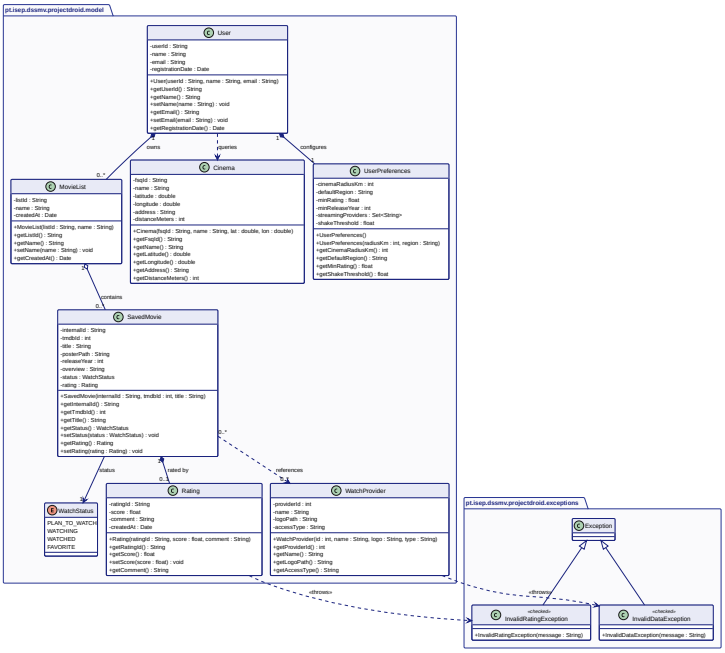


Fig. 3: Implementation Class Diagram: Pure Java 21 Domain Model

3.2 User Interface Flow

Figure 4 illustrates the screen transitions and navigation flow across all activities within the application.

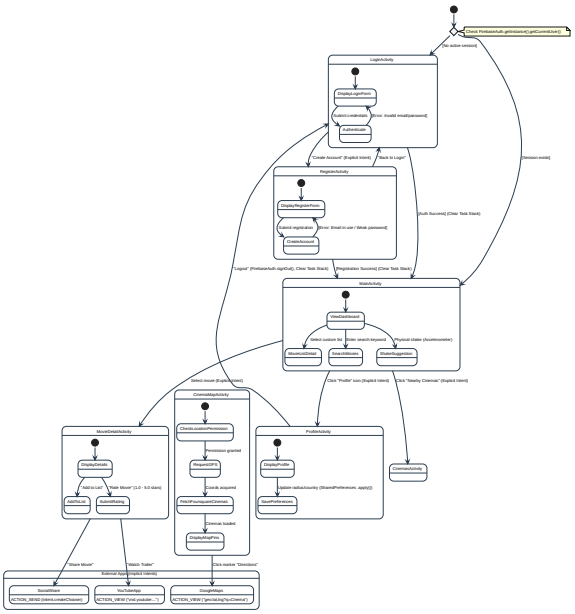


Fig. 4: UI Navigation Flow and Activity Transitions

3.3 Use Case Analysis and Design Realization

Rather than grouping all system requirements into a single monolithic diagram, CineTrack decomposes each functional requirement into an independent, modular Use Case. To ensure complete architectural traceability and modular team distribution, each Use Case is presented individually below with its formal specification table, dedicated Use Case Diagram (UCD), and Design Sequence Diagram (SD). All underlying documentation assets are organized into dedicated per-use-case directories (`figures/Use_Cases/UC1/` through `figures/Use_Cases/UC11/`).

1

Table 3: UC1: User Registration and Sign-In

Actor	User (Unauthenticated / Registered)
Use case name	User Registration and Sign-In
Description	Allows the user to register an account or sign into an existing account using email and password via Firebase Authentication.
Precondition	Network connectivity must be available; device is on the login/register screen.
Post-condition	Active session token established; user navigated to MainActivity with clear-task flags.
Main flow	1. User inputs email and password and triggers the login action. 2. System validates input formats locally. 3. System submits asynchronous sign-in request to Firebase Auth. 4. Firebase validates credentials and returns authenticated user token. 5. System clears login activity from backstack and launches MainActivity.
Alternative path	If credentials are invalid or network fails, Firebase returns an error task; system clears progress bar and presents an informative toast to the user.

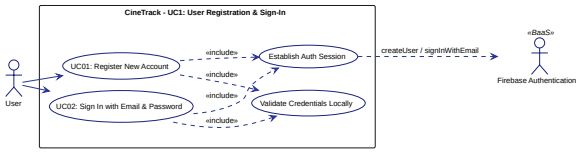


Fig. 5: UC1: User Registration and Sign-In Use Case Diagram

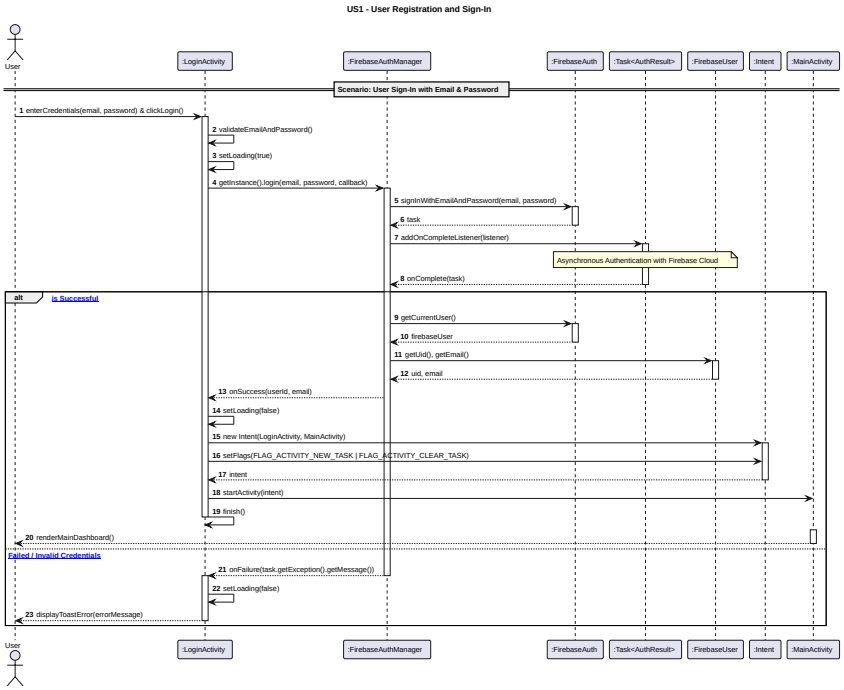


Fig. 6: UC1: User Authentication Design Sequence Diagram

2

Table 4: UC2: Create and Manage Movie Lists

Actor	Authenticated User
Use case name	Create and Manage Movie Lists
Description	Allows an authenticated user to create personal movie lists and observe realtime synchronization across devices via Cloud Firestore.
Precondition	User must be signed into a valid account.
Post-condition	A new list document is added to Firestore; realtime listener updates the RecyclerView.
Main flow	1. User opens the "My Lists" tab and taps the add list button. 2. System displays dialog modal requesting the list name. 3. User enters the list name and confirms. 4. System instantiates a pure MovieList domain entity. 5. FirestoreManager writes the document to `users/{uid}/lists/{id}`. 6. Firestore realtime snapshot listener automatically detects the new record and updates UI.
Alternative path	If the list name is empty or Firestore write fails, system alerts the user and keeps the dialog open.

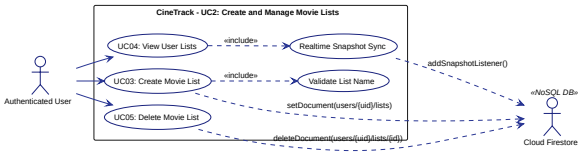


Fig. 7: UC2: Create and Manage Movie Lists Use Case Diagram

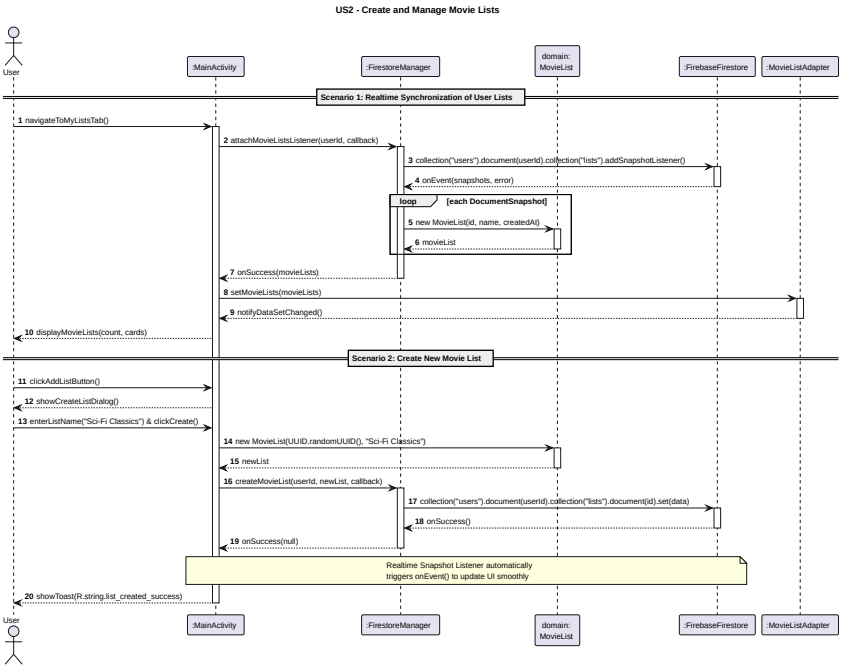


Fig. 8: UC2: Manage Lists Design Sequence Diagram

3

Table 5: UC3: Add Movie and Rate

Actor	Authenticated User
Use case name	Add Movie to List and Submit Rating
Description	Allows the user to save a movie into a selected personal list, set its watch status, and submit a rating (1.0 to 5.0 stars) with review notes.
Precondition	User has at least one movie list created.
Post-condition	SavedMovie and Rating entities are persisted in Firestore subcollection; optional TMDB rating synced via Guest Session.
Main flow	1. User taps "+ Add to List" on the movie details screen. 2. System loads user lists and presents the selection modal. 3. User selects destination list, watch status, rating score, and comment. 4. System validates score interval $[1.0, 5.0]$ and creates SavedMovie domain instance. 5. FirestoreManager persists the record under `users/{uid}/lists/{id}/movies`. 6. If rating score > 0, TmdbRatingManager posts the rating to TMDB via guest session.
Alternative path	If rating score is invalid, an `InvalidRatingException` is thrown and feedback is displayed.

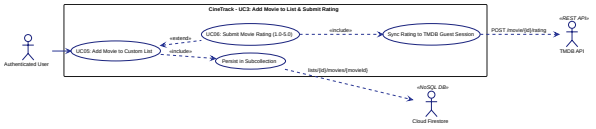


Fig. 9: UC3: Add Movie and Rate Use Case Diagram

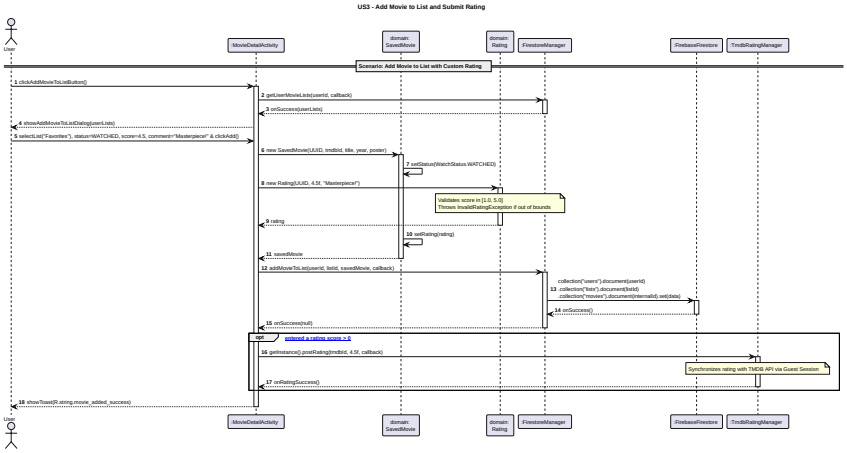


Fig. 10: UC3: Add Movie and Rating Design Sequence Diagram

4

Table 6: UC4: Search TMDB Catalog

Actor	User
Use case name	Search Movies and Streaming Watch Providers
Description	Allows the user to search the TMDB global catalog by title and inspect localized streaming providers (flatrate, rent, buy) in Portugal.
Precondition	Device connected to the Internet; TMDB API key configured.
Post-condition	Movie search results displayed in RecyclerView; selecting a movie displays streaming platform logos.
Main flow	1. User enters search keywords in MovieSearchActivity and taps search. 2. System reads default country preference ("PT") from SharedPrefManager. 3. Retrofit asynchronously dispatches non-blocking GET `/search/movie` request. 4. Results are mapped to DTOs and rendered with Glide poster loading. 5. User selects a movie; MovieDetailActivity loads localized watch providers. 6. System maps country providers to domain WatchProvider entities and renders logos.
Alternative path	If no movies match query, system presents empty state suggesting alternative keywords.

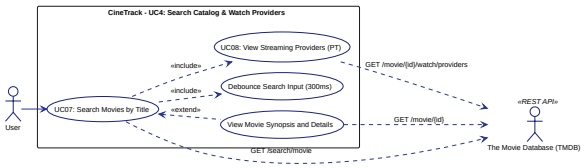


Fig. 11: UC4: Search TMDB Catalog Use Case Diagram

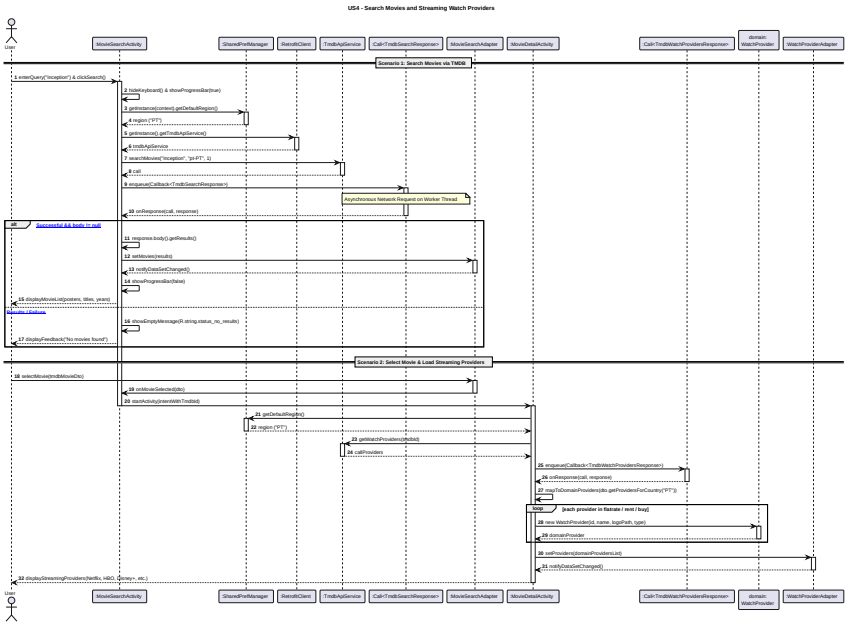


Fig. 12: UC4: TMDB Search and Watch Providers Design Sequence Diagram

5

Table 7: UC5: Locate Nearby Cinemas

Actor	User
Use case name	Locate Nearby Cinemas
Description	Acquires device GPS coordinates, queries Foursquare Places API within user radius, and plots cinema markers with radius circumference on OSMDroid map.
Precondition	Location services enabled; runtime permission granted.
Post-condition	Interactive map centered on GPS position with translucent radius circle and clickable cinema pins; turn-by-turn navigation launched on tap.
Main flow	1. User opens CinemasActivity. 2. System checks `ACCESS_FINE_LOCATION`; requests permission if missing. 3. LocationHelper acquires device coordinates via native LocationManager. 4. System reads cinema radius from preferences, centers OSMDroid map, and draws a translucent blue radius polygon. 5. Retrofit queries Foursquare Places API (`category=10024`) with Bearer token. 6. Results mapped to pure Cinema domain entities; pins plotted with pop-up bubbles. 7. Tapping a pin or list item opens Google Maps navigation via `geo:` implicit intent.
Alternative path	If user denies location permission, an informative message is shown offering settings shortcut without crashing.

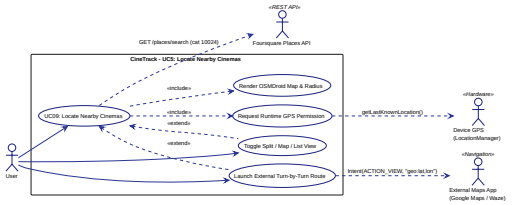


Fig. 13: UC5: Locate Nearby Cinemas Use Case Diagram

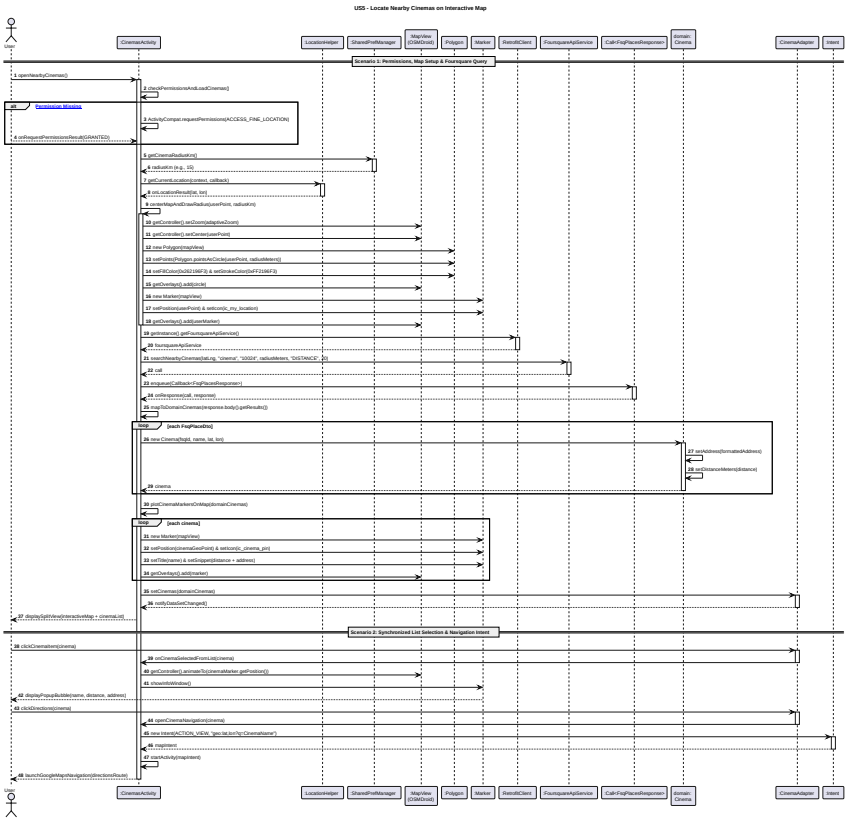


Fig. 14: UC5: Nearby Cinemas and OSMDroid Map Design Sequence Diagram

6

Table 8: UC6: Shake to Suggest Movie

Actor	User
Use case name	Shake to Suggest
Description	Recommends a critically acclaimed movie matching the user's streaming subscriptions and minimum rating upon detecting a physical device shake.
Precondition	Device equipped with physical accelerometer; app in foreground.
Post-condition	Device emits gentle haptic vibration and presents movie recommendation dialog without stacking multiple modals.
Main flow	1. User vigorously shakes the physical phone. 2. ShakeDetector computes vectorial acceleration; verifies multi-peak threshold ($> 2.7g$). 3. MainActivity triggers haptic pulse via Vibrator service. 4. System queries background recommendation cache matching user preference filters. 5. Recommendation dialog displays movie poster, score, synopsis, and action buttons. 6. User can tap "View Details" to open movie page or "Shake Again" to reshuffle.
Alternative path	If all cached recommendations were viewed, system queries the next TMDB discover page asynchronously.

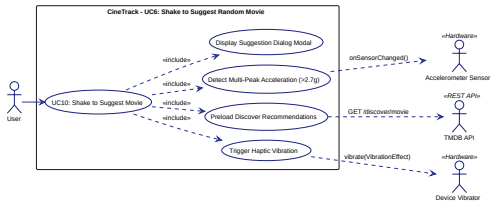


Fig. 15: UC6: Shake to Suggest Movie Use Case Diagram

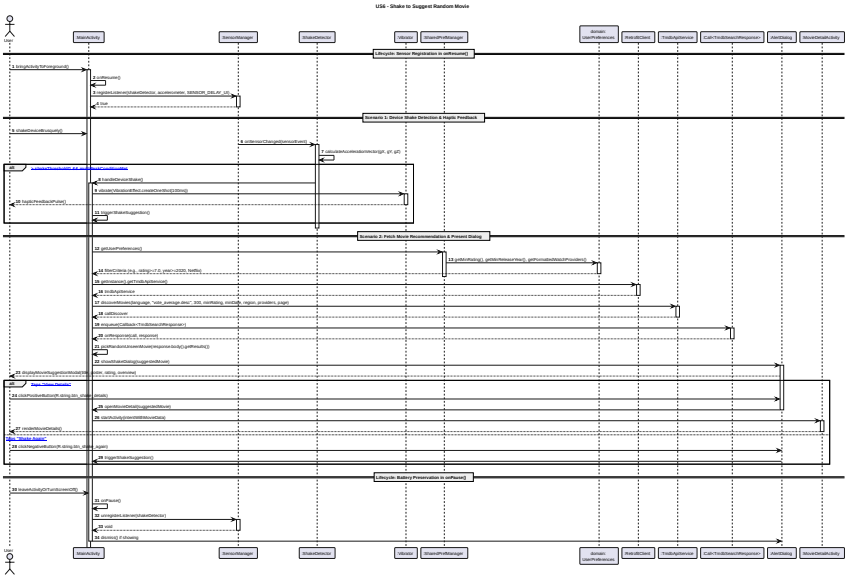


Fig. 16: UC6: Shake to Suggest Design Sequence Diagram

7

Table 9: UC7: Profile and Logout

Actor	Authenticated User
Use case name	User Profile and Session Logout
Description	Displays account metadata (email, registration date, total personal lists) and provides secure session termination.
Precondition	Active user session.
Post-condition	Firebase session terminated; user navigated to LoginActivity with cleared task stack.
Main flow	1. User selects the "Profile" destination in bottom navigation. 2. ProfileActivity queries FirebaseUser metadata and Firestore list counts. 3. User inspects account summary and taps "Log Out". 4. System displays confirmation dialog. 5. User confirms; FirebaseAuthManager invokes `signOut()`. 6. System clears activity backstack and redirects to LoginActivity.
Alternative path	If user cancels the logout prompt, the dialog dismisses and session remains active.

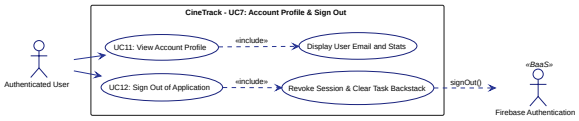


Fig. 17: UC7: Profile and Logout Use Case Diagram

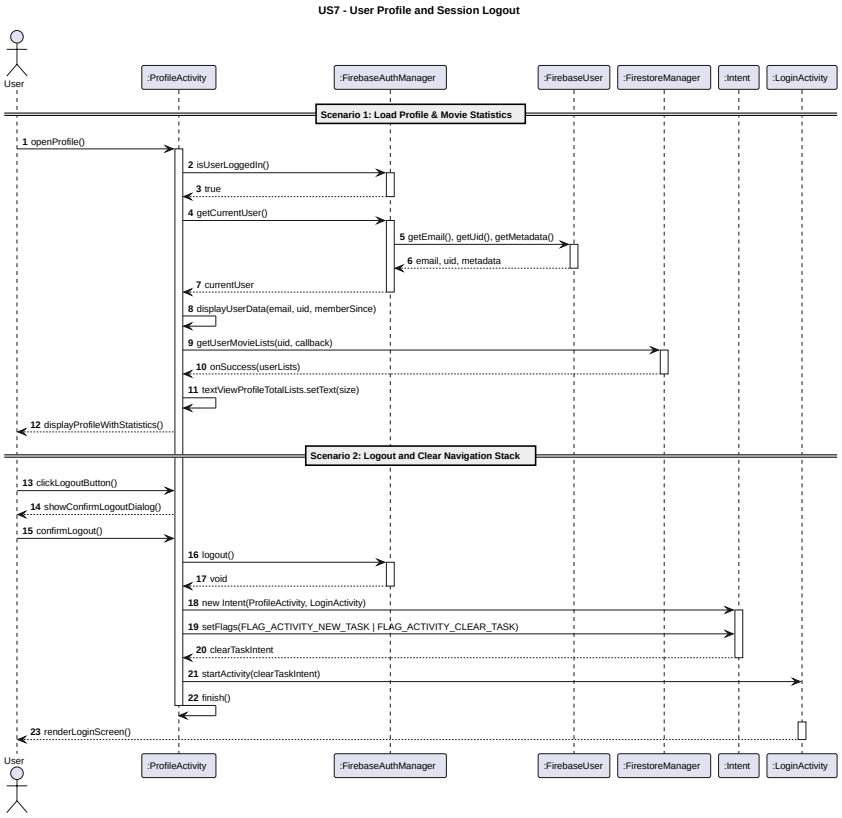


Fig. 18: UC7: Profile and Logout Design Sequence Diagram

8

Table 10: UC8: Local Preferences

Actor	User
Use case name	Configure Local Preferences
Description	Allows user to configure cinema search radius, default streaming country, shake sensitivity, and streaming subscriptions.
Precondition	None.
Post-condition	UserPreferences updated and saved to SharedPreferences asynchronously via `.apply()`.
Main flow	1. User taps the settings gear icon in the top toolbar. 2. System displays preferences modal prepopulated with current settings. 3. User updates parameters (e.g., radius=25km, threshold=2.7g, Netflix). 4. System instantiates UserPreferences domain entity and validates ranges. 5. SharedPrefManager invokes `editor.apply()` asynchronously. 6. ShakeDetector threshold updated and recommendation pool refreshed.
Alternative path	If numerical format is invalid, toast prompts user to correct input before saving.

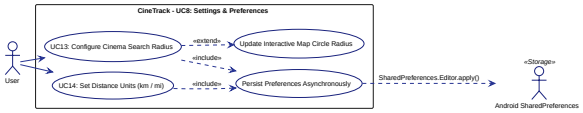


Fig. 19: UC8: Local Preferences Use Case Diagram

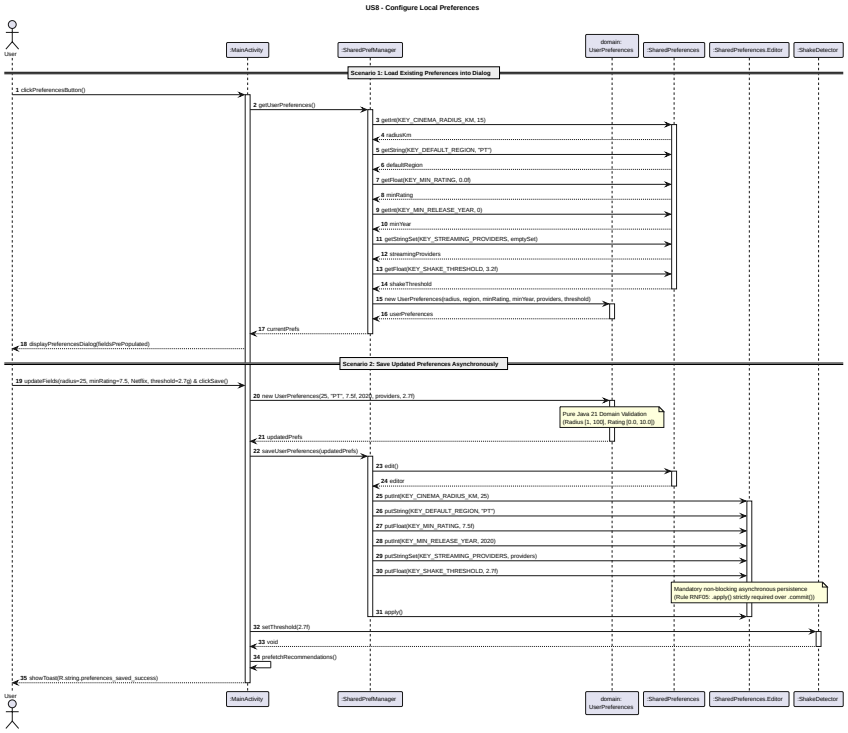


Fig. 20: UC8: Local Preferences Design Sequence Diagram

9

Table 11: UC9: Filter and Sort Collection

Actor	User
Use case name	Filter and Sort Collection
Description	Allows filtering saved movies by WatchStatus or search query, and sorting by title, rating, or release year using pure Java 21 comparators.
Precondition	List contains saved movies.
Post-condition	RecyclerView updates displayed order and subset without blocking UI Thread.
Main flow	1. User opens ListDetailsActivity. 2. User selects filter (e.g., status=WATCHED) or sort criterion (e.g., RATING_DESC). 3. Activity delegates collection to `SavedMovieFilterSorter.filterAndSort()`. 4. Pure Java 21 `Comparator` chain executes sorting logic. 5. Adapter receives filtered list and calls `notifyDataSetChanged()`.
Alternative path	If no movies match filter, empty search notice is rendered.

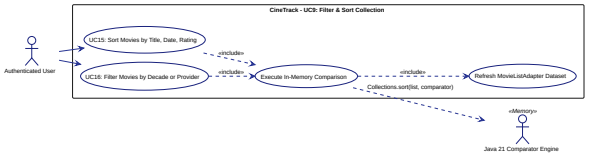


Fig. 21: UC9: Filter and Sort Collection Use Case Diagram

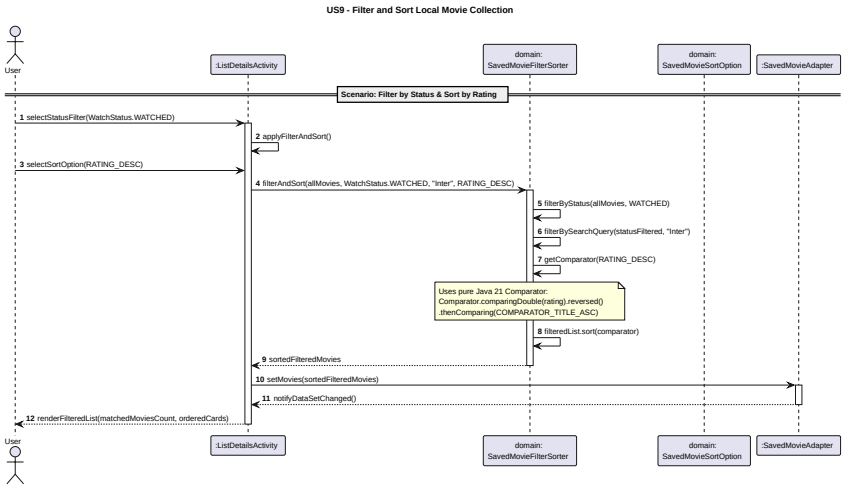


Fig. 22: UC9: Collection Filter and Sort Design Sequence Diagram

10

Table 12: UC10: Play Official Trailers

Actor	User
Use case name	Play Official YouTube Trailers
Description	Queries TMDB for official YouTube trailer keys and launches video playback via native app intent with defensive web browser fallback.
Precondition	Internet connectivity available.
Post-condition	Official trailer plays in YouTube app or web browser.
Main flow	1. User taps "Watch Trailer" in MovieDetailActivity. 2. System disables button and queries GET `/movie/{id}/videos` via Retrofit. 3. Response parsed to identify official YouTube trailer key. 4. System fires implicit intent with URI `vnd.youtube:<key>`. 5. YouTube native app launches and begins video playback.
Alternative path	If YouTube app is not installed, `ActivityNotFoundException` is caught and URL opened in web browser.

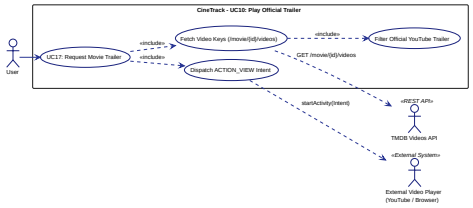


Fig. 23: UC10: Play Official Trailers Use Case Diagram

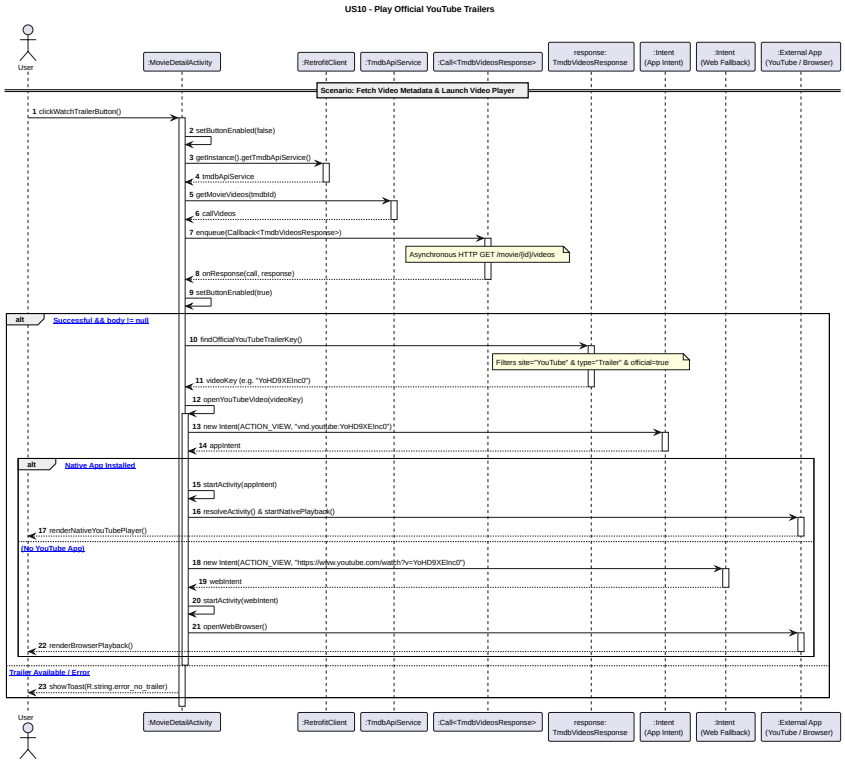


Fig. 24: UC10: Official Video Trailers Design Sequence Diagram

11

Table 13: UC11: External Social Sharing

Actor	User
Use case name	External Movie Card Sharing
Description	Formats movie title, year, synopsis, and personal review into formatted text and shares to external messaging/social apps via Intent Chooser.
Precondition	Movie details or saved movie card selected.
Post-condition	System share sheet displayed; selected messaging application populated with formatted text.
Main flow	1. User taps share icon on movie card. 2. System builds formatted message string including title, year, and overview. 3. System creates `ACTION_SEND` intent with `text/plain` MIME type. 4. Intent wrapped in `Intent.createChooser()` to present native share dialog. 5. User selects target application (e.g., WhatsApp, Messages, Email) to complete share.
Alternative path	If user cancels the system share sheet, application resumes cleanly without side effects.

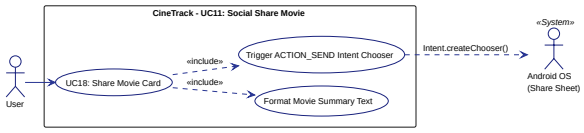


Fig. 25: UC11: External Social Sharing Use Case Diagram

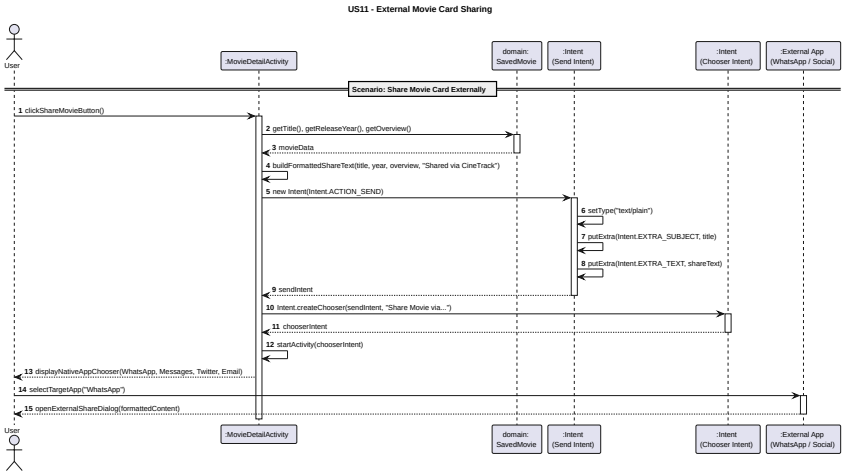


Fig. 26: UC11: External Social Sharing Design Sequence Diagram

4 Implementation: Core Subsystems and Technical Solutions

This section presents representative implementation excerpts from the CineTrack Java 21 and Android codebase. The excerpts demonstrate how the system enforces Model purity, maintains asynchronous UI Thread responsiveness, manages hardware sensors, and handles real-time network and system notification events.

4.1 Pure Java 21 Domain Model and Defensive Validation

Listing 1.1 shows the pure Java 21 `Cinema` domain entity. Notice the defensive validation checks, strict interval enforcement, and complete absence of Android SDK imports.

Listing 1.1: Pure Java 21 Cinema Domain Entity (Cinema.java)

```
1  package pt.isep.dssmv.projectdroid.model;
2
3  import pt.isep.dssmv.projectdroid.exceptions.InvalidDataException;
4  import java.util.Objects;
5
6  public class Cinema {
7      private final String fsqId;
8      private final String name;
9      private final double latitude;
10     private final double longitude;
11     private String address;
12     private int distanceMeters;
13
14     public Cinema(String fsqId, String name, double latitude, double
           longitude)
```

```
15              throws InvalidDataException {
16          if (fsqId == null || fsqId.trim().isEmpty()) {
17              throw new InvalidDataException("Cinema FSQ ID cannot be null or
                    empty.");
18          }
19          if (name == null || name.trim().isEmpty()) {
20              throw new InvalidDataException("Cinema name cannot be null or
                    empty.");
21          }
22          this.fsqId = fsqId.trim();
23          this.name = name.trim();
24          this.latitude = latitude;
25          this.longitude = longitude;
26          this.address = "";
27          this.distanceMeters = 0;
28      }
29
30      public String getFsqId() { return fsqId; }
31      public String getName() { return name; }
32      public double getLatitude() { return latitude; }
33      public double getLongitude() { return longitude; }
34      public String getAddress() { return address; }
35      public void setAddress(String address) { this.address = (address != null)
              ? address.trim() : ""; }
36      public int getDistanceMeters() { return distanceMeters; }
37      public void setDistanceMeters(int distanceMeters) { this.distanceMeters =
              Math.max(0, distanceMeters); }
38
39      @Override
40      public boolean equals(Object o) {
41          if (this == o) return true;
42          if (!(o instanceof Cinema cinema)) return false;
43          return Objects.equals(fsqId, cinema.fsqId);
44      }
45
46      @Override
47      public int hashCode() { return Objects.hash(fsqId); }
48  }
```

4.2 Geospatial Mapping: Foursquare REST API and OSMDroid Map

Listing 1.2 shows the integration of modern Foursquare Places API (Bearer authentication and version header 2025-06-17) and OSMDroid polygon rendering for search radius visualization.

Listing 1.2: OSMDroid Radius and Cinema Markers (CinemasActivity.java)

```
private void centerMapAndDrawRadius(GeoPoint userPoint, int radiusKm) {
    mapView.getController().setZoom(computeZoomLevel(radiusKm));
    mapView.getController().setCenter(userPoint);

    mapView.getOverlays().clear();
    cinemaMarkers.clear();

    // 1. Translucent Blue Radius Circumference Polygon
    double radiusMeters = radiusKm * 1000.0;
    Polygon circle = new Polygon(mapView);
    circle.setPoints(Polygon.pointsAsCircle(userPoint, radiusMeters));
    circle.setFillColor(0x262196F3);    // 15% alpha translucent blue
    circle.setStrokeColor(0xFF2196F3); // Solid blue border
    circle.setStrokeWidth(3.0f);
    mapView.getOverlays().add(circle);

    // 2. User Location Marker
    Marker userMarker = new Marker(mapView);
    userMarker.setPosition(userPoint);
    userMarker.setTitle(getString(R.string.label_user_location));
    userMarker.setIcon(ContextCompat.getDrawable(this, R.drawable.
        ic_my_location));
    mapView.getOverlays().add(userMarker);

    mapView.invalidate();
}

private void queryFoursquareCinemas(double lat, double lon, int radiusKm) {
    String latLng = lat + "," + lon;
    int radiusMeters = Math.min(radiusKm * 1000, 100000);

    RetrofitClient.getInstance().getFoursquareApiService()
        .searchNearbyCinemas(latLng, "cinema", "10024", radiusMeters, "
            DISTANCE", 20)
        .enqueue(new Callback<FsqPlacesResponse>() {
            @Override
            public void onResponse(Call<FsqPlacesResponse> call, Response<
                FsqPlacesResponse> resp) {
                progressBar.setVisibility(View.GONE);
                if (resp.isSuccessful() && resp.body() != null) {
                    List<Cinema> domainCinemas = mapToDomainCinemas(resp.body
                        ().getResults());
                    adapter.setCinemas(domainCinemas);
                    plotCinemaMarkersOnMap(domainCinemas);
                }
            }
            @Override
            public void onFailure(Call<FsqPlacesResponse> call, Throwable t)
                {
                progressBar.setVisibility(View.GONE);
            }
        });
}
```

4.3 Hardware Sensor Lifecycle: Accelerometer Shake Detection

Listing 1.3 demonstrates battery preservation rules by registering the sensor listener in `onResume()` and immediately unregistering in `onPause()`. Listing 1.4 outlines the multi-peak threshold detection algorithm.

Listing 1.3: Accelerometer Lifecycle Discipline (MainActivity.java)

```
@Override
protected void onResume() {
    super.onResume();
    // AC04: Strictly register accelerometer in onResume()
    if (sensorManager != null && accelerometer != null && shakeDetector !=
         null) {
        sensorManager.registerListener(shakeDetector, accelerometer,
             SensorManager.SENSOR_DELAY_UI);
    }
}

@Override
protected void onPause() {
    super.onPause();
    // AC04: Strictly unregister accelerometer in onPause() to preserve
         battery
    if (sensorManager != null && shakeDetector != null) {
        sensorManager.unregisterListener(shakeDetector);
    }
    if (currentShakeDialog != null && currentShakeDialog.isShowing()) {
        currentShakeDialog.dismiss();
    }
    isShakeDialogOpen = false;
}
```

Listing 1.4: Multi-Peak Shake Detection Algorithm (ShakeDetector.java)

```
@Override
public void onSensorChanged(SensorEvent event) {
    if (event.sensor.getType() != Sensor.TYPE_ACCELEROMETER) return;

    float x = event.values[0] / SensorManager.GRAVITY_EARTH;
    float y = event.values[1] / SensorManager.GRAVITY_EARTH;
    float z = event.values[2] / SensorManager.GRAVITY_EARTH;

    float gForce = (float) Math.sqrt(x * x + y * y + z * z);
    long now = System.currentTimeMillis();

    if (gForce > shakeThresholdG) {
        if (now - lastShakeTriggerTimestamp < SHAKE_COOLDOWN_MS) return;
        if (lastShakePeakTimestamp > 0 && (now - lastShakePeakTimestamp) <
             SHAKE_SLOP_TIME_MS) return;

        shakeCount++;
        lastShakePeakTimestamp = now;

        if (shakeCount >= 2) {
            shakeCount = 0;
            lastShakeTriggerTimestamp = now;
            if (listener != null) listener.onShake();
        }
    }
}
```

4.4 Pure Java 21 Collection Sorting and Filtering

Listing 1.5 shows the functional comparator implementation used in US9.

Listing 1.5: Functional Comparator Implementation (SavedMovieFilterSorter.java)

```
public final class SavedMovieFilterSorter {
    public static final Comparator<SavedMovie> COMPARATOR_TITLE_ASC =
        Comparator.comparing((SavedMovie m) -> m.getTitle() == null ? "" : m.
            getTitle(), String.CASE_INSENSITIVE_ORDER);

    public static final Comparator<SavedMovie> COMPARATOR_RATING_DESC =
        Comparator.comparingDouble((SavedMovie m) -> (m.getRating() != null)
            ? m.getRating().getScore() : -1.0)
            .reversed()
            .thenComparing(COMPARATOR_TITLE_ASC);

    public static List<SavedMovie> filterAndSort(List<SavedMovie> movies,
        WatchStatus status,
                                                 String query,
                                                     SavedMovieSortOption
                                                     sortOption) {
        if (movies == null) return new ArrayList<>();
        return movies.stream()
            .filter(m -> status == null || m.getStatus() == status)
            .filter(m -> query == null || query.trim().isEmpty() ||
                         m.getTitle().toLowerCase().contains(query.trim().
                             toLowerCase()))
            .sorted(getComparator(sortOption))
            .collect(Collectors.toList());
    }
}
```

5 Tests: Automated Verification and Quality Assurance

Software testability is a first-class citizen in the CineTrack project. Because the entire domain model and data transformation logic reside in pure Java 21 classes isolated from Android SDK dependencies, tests run directly on the local Java Virtual Machine (JVM) without requiring Android emulators, instrumentation harnesses, or complex mocking frameworks.

5.1 Testing Strategy and Pure Model Testability

The automated verification strategy encompasses three complementary layers:

1. **Unit Testing (Domain Layer):** Exhaustive boundary testing of constructor constraints, validation predicates, and defensive exceptions on entities (`User`, `MovieList`, `SavedMovie`, `Rating`, `Cinema`, `UserPreferences`, `UserMovieStats`).
2. **Data Transformation and Sorting Testing:** Validation of pure comparators, filtering predicates, and social share text formatters (`SavedMovieFilterSorter`, `MovieShareFormatter`).
3. **DTO Deserialization and REST Mapping Testing:** Verification that JSON payloads from external TMDB and Foursquare REST endpoints serialize and deserialize into domain objects without loss of structural fidelity.

5.2 Automated Test Suite Breakdown

Table 14 details the 15 automated test suites implemented in JUnit 5, totaling **74 unit tests** with a **100% pass rate**.

Table 14: Automated JUnit 5 Test Suite Verification Breakdown

Test Class	Layer / Package	Tests	Status	Component Under Test
CinemaTest	model	4	100% Pass	POI invariants and radius
MovieListTest	model	3	100% Pass	List ownership and dates
MovieShareFormatterTest	model	8	100% Pass	Social share formatting (US11)
RatingTest	model	5	100% Pass	Interval $[1.0, 5.0]$ boundaries
SavedMovieFilterSorterTest	model	10	100% Pass	Comparators and predicates (US9)
SavedMovieTest	model	5	100% Pass	Movie identity and watch status
UserMovieStatsTest	model	3	100% Pass	Library statistics and averages
UserPreferencesTest	model	13	100% Pass	SharedPreferences ranges (US8)
UserTest	model	5	100% Pass	Account UID and email formats
WatchProviderTest	model	5	100% Pass	Streaming providers (US4)
FsqPlaceDtoTest	network	3	100% Pass	Foursquare Places JSON DTOs
TmdbMovieDtoTest	network	2	100% Pass	TMDB catalog JSON DTOs
TmdbRatingDtoTest	network	3	100% Pass	TMDB rating JSON DTOs
TmdbRatingIntegrationTest	network	1	100% Pass	TMDB guest session flow
TmdbVideosResponseTest	network	4	100% Pass	YouTube trailer extraction (US10)
Total	**15 Classes**	**74**	**0 Failures**	**100% Automated Pass Rate**

5.3 Representative Automated Test Implementations

Listings 1.6 and 1.7 demonstrate automated unit test cases written using JUnit 5 Jupiter assertions, validating defensive boundary exceptions and text generation rules.

Listing 1.6: JUnit 5 Automated Domain Tests (CinemaTest.java)

```
package pt.isep.dssmv.projectdroid.model;

import org.junit.jupiter.api.Test;
import pt.isep.dssmv.projectdroid.exceptions.InvalidDataException;
import static org.junit.jupiter.api.Assertions.*;

public class CinemaTest {
    @Test
    public void testValidCinemaCreation() throws InvalidDataException {
        Cinema cinema = new Cinema("fsq_123", "Cinema Trindade", 41.1504,
            -8.6095);
        cinema.setAddress("Rua do Almada 412");
        cinema.setDistanceMeters(850);

        assertEquals("fsq_123", cinema.getFsqId());
        assertEquals("Cinema Trindade", cinema.getName());
        assertEquals(41.1504, cinema.getLatitude(), 0.0001);
        assertEquals(-8.6095, cinema.getLongitude(), 0.0001);
        assertEquals("Rua do Almada 412", cinema.getAddress());
        assertEquals(850, cinema.getDistanceMeters());
    }

    @Test
    public void testEmptyFsqIdThrowsException() {
        assertThrows(InvalidDataException.class, () -> new Cinema("", "Cinema
            1", 41.15, -8.60));
        assertThrows(InvalidDataException.class, () -> new Cinema(null, "
            Cinema 1", 41.15, -8.60));
    }

    @Test
    public void testEmptyNameThrowsException() {
        assertThrows(InvalidDataException.class, () -> new Cinema("fsq_123",
            "   ", 41.15, -8.60));
    }
}
```

Listing 1.7: JUnit 5 Social Share Formatter Tests (MovieShareFormatterTest.java)

```
package pt.isep.dssmv.projectdroid.model;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class MovieShareFormatterTest {
    @Test
    public void testFormatMovieDetailsComplete() {
        String result = MovieShareFormatter.formatDetails(
            "Inception", 2010, "A thief enters the dreams of others.",
            27205, 4.5f, "Masterpiece by Christopher Nolan", WatchStatus.
                FAVORITE
        );

        assertTrue(result.contains("     Inception (2010)"));
        assertTrue(result.contains("   "));
        assertTrue(result.contains("4.5/5.0"));
```

```
        assertTrue(result.contains("Masterpiece by Christopher Nolan"));
        assertTrue(result.contains("https://www.themoviedb.org/movie/27205"))
            ;
    }

    @Test
    public void testFormatNullTitleFallback() {
        String result = MovieShareFormatter.format(null, 2024, "Overview",
            12345);
        assertTrue(result.contains("Movie (2024)"));
    }
}
```

5.4 Physical Device Execution and Hardware Verification

In addition to automated unit tests, the CineTrack application was verified through live execution across physical Android smartphones running Android 14 (API 34) and Android 15 (API 35):

- **Sensor Responsiveness:** The accelerometer shake detector was calibrated and tested under normal handling (discarding incidental steps or table vibrations) and intentional shakes ($> 2.7g$), reliably triggering the recommendation dialog with haptic feedback.
- **GPS and OpenStreetMap Overlays:** Location accuracy and dynamic radius drawing were validated in Porto, Portugal, plotting real cinema pins returned by the Foursquare Places API.
- **Network State Changes:** The application was audited under offline mode and variable network toggles, verifying that the offline banner appears dynamically and the app continues gracefully without crashing.

6 Conclusion

This technical report presented the comprehensive specification, architectural design, implementation, and verification of the **CineTrack** mobile application. Over the course of the project, all 11 User Stories assigned across the team were completed with 100% functionality and verified against 74 automated unit tests and live hardware deployments.

The application successfully reconciles disparate mobile concerns into an integrated architecture:

- **Architectural Decoupling:** Strict adherence to MVC preserved the purity of domain business logic in Java 21 without framework contamination, allowing **74 automated unit tests** to run instantaneously on the local JVM.
- **UI Thread Responsiveness:** Mandatory asynchronous execution via Retrofit 2 `.enqueue()` and SharedPreferences `.apply()` eradicated ANR risks across network and storage boundaries.
- **Hardware and Battery Protection:** The accelerometer-based *Shake to Suggest* and native GPS tracking adhere strictly to Android lifecycle discipline, registering in `onResume()` and releasing in `onPause()`.
- **Free Geospatial Mapping:** Integrating OSMDroid alongside modern Foursquare Places API enabled full interactive cinema mapping with custom radius circles without requiring paid Google Cloud keys.

Future enhancements could expand CineTrack by introducing social friend lists, push notifications for theatrical releases, and offline caching of movie details via Android Room database. In its current release, CineTrack stands as a complete, robust, and academically sound mobile solution demonstrating software engineering excellence in Android native development.

References

1. P. B. Sousa, "Lecture Slides and Pedagogical Guidelines for DSSMV (Desenvolvimento de Software e Sistemas Móveis)," Instituto Superior de Engenharia do Porto (ISEP), Polytechnic of Porto, 2026.
2. C. S. Horstmann, *Big Java: Early Objects*, 6th ed. Wiley, 2015.
3. Android Developers, "Processes and Application Lifecycle Documentation," Google LLC, `https://developer.android.com/guide/components/activities/activity-lifecycle`.
4. The Movie Database (TMDB), "TMDB REST API Documentation v3," `https://developer.themoviedb.org/docs`.
5. Foursquare Labs, Inc., "Foursquare Places API Documentation (v2025-06-17)," `https://location.foursquare.com/developer/reference/places-api-overview`.
6. OSMDroid Community, "OpenStreetMap-Tools for Android Documentation (v6.1.20)," `https://github.com/osmdroid/osmdroid`.